Presentation: Road Accidents in France

A MLOps Architecture for Road Safety Analytics

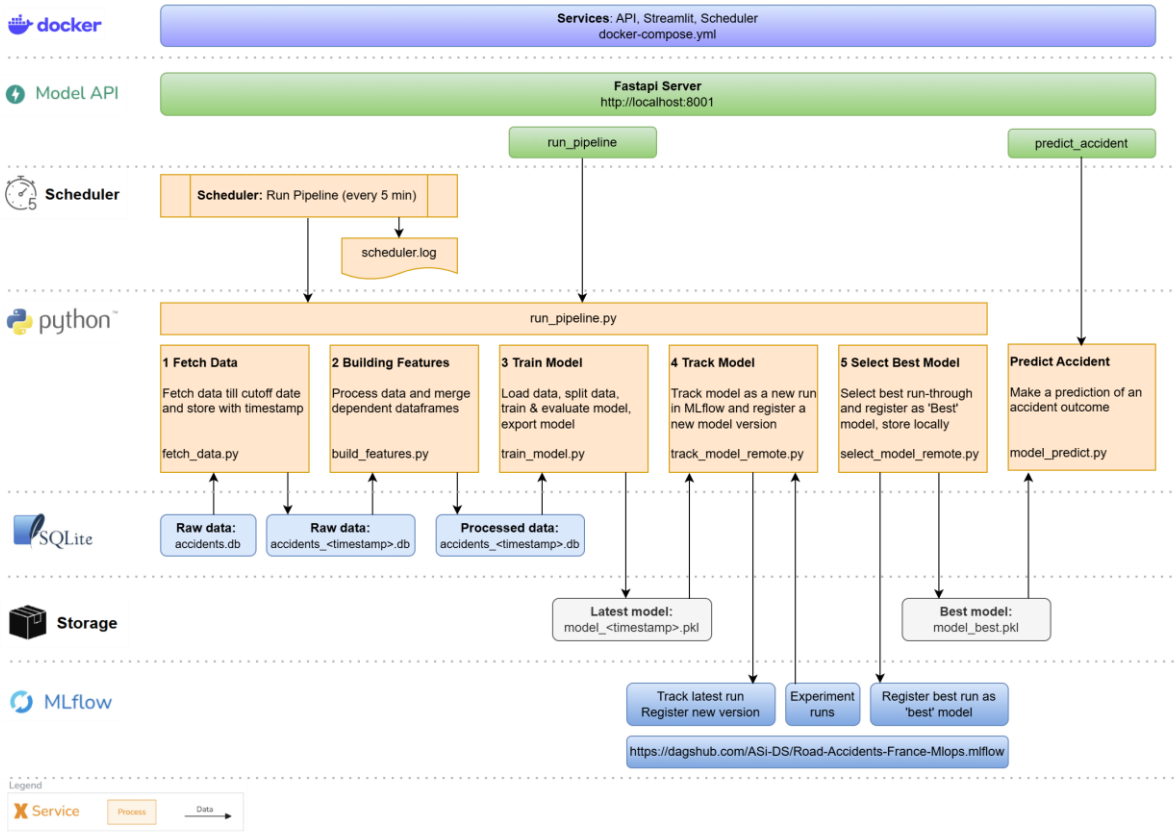
Objective: Building a MLOps framework to predict accident severity.
Focus: Integration of scheduling, experiment tracking, and containerizing for a multi-class classification model.

Team: Jérôme Morin-Drouin, Alke Simmler
Mentor: Nicolas Fradin

Content Table

1 Architecture Diagram 3
2 Components 4
3 Scheduler and Pipeline 5
4 Data loading and processing 6
5 Data Processing Diagram: 7
6 MLflow with DagsHub: Experiment tracking & Model versioning 8
7 Accident Prediction 9
8 FastAPI Server Interface 10
9 Docker 11
10 Next Steps & Future Enhancements 12

1 Architecture Diagram



Project source files are available on github:
https://github.com/jmdrouin/Road-Accidents-France-Mlops.git

2 Components

docker	**Docker**: Isolation and virtualization • **Standardization**: Guarantees a consistent environment across development, testing, and production. • **Encapsulation**: Packages the ML model, code, and specific library versions into a portable unit. • **Efficiency**: Provides process-level isolation that is faster and uses fewer resources than Virtual Machines.
FastAPI	**FastAPI**: Deployment in a microservice architecture • **Deployment**: Wraps service methods into a web API providing endpoints for clients to interact with the model. • **Decoupling**: Provides an API contract decoupling the service from clients. • **Security**: Provides authentication and authorization (not implemented).
mlflow DagsHub	**MLflow with DagsHub**: Remote experiment tracking & model versioning • **Experiment Tracking**: Logs parameters, metrics, and artifacts during training runs. • **Model Registry**: Serves as a central hub to version models. • **Access Control**: Provides built-in, team-based access controls. • **DagsHub Server**: Integrates Git and provides a hosted, remote MLflow tracking server offering a web-based UI to visualize MLflow experiment runs side-by-side.
Scheduler	**Scheduler**: Automated workflow • **Automation of Pipeline**: Executes ETL (Extract, Transform, Load), model training, tracking, and evaluation at specific intervals • **Dependency Management**: Ensures tasks happen in a defined order • **Error handling and logging**: Encapsulates error handling and logging

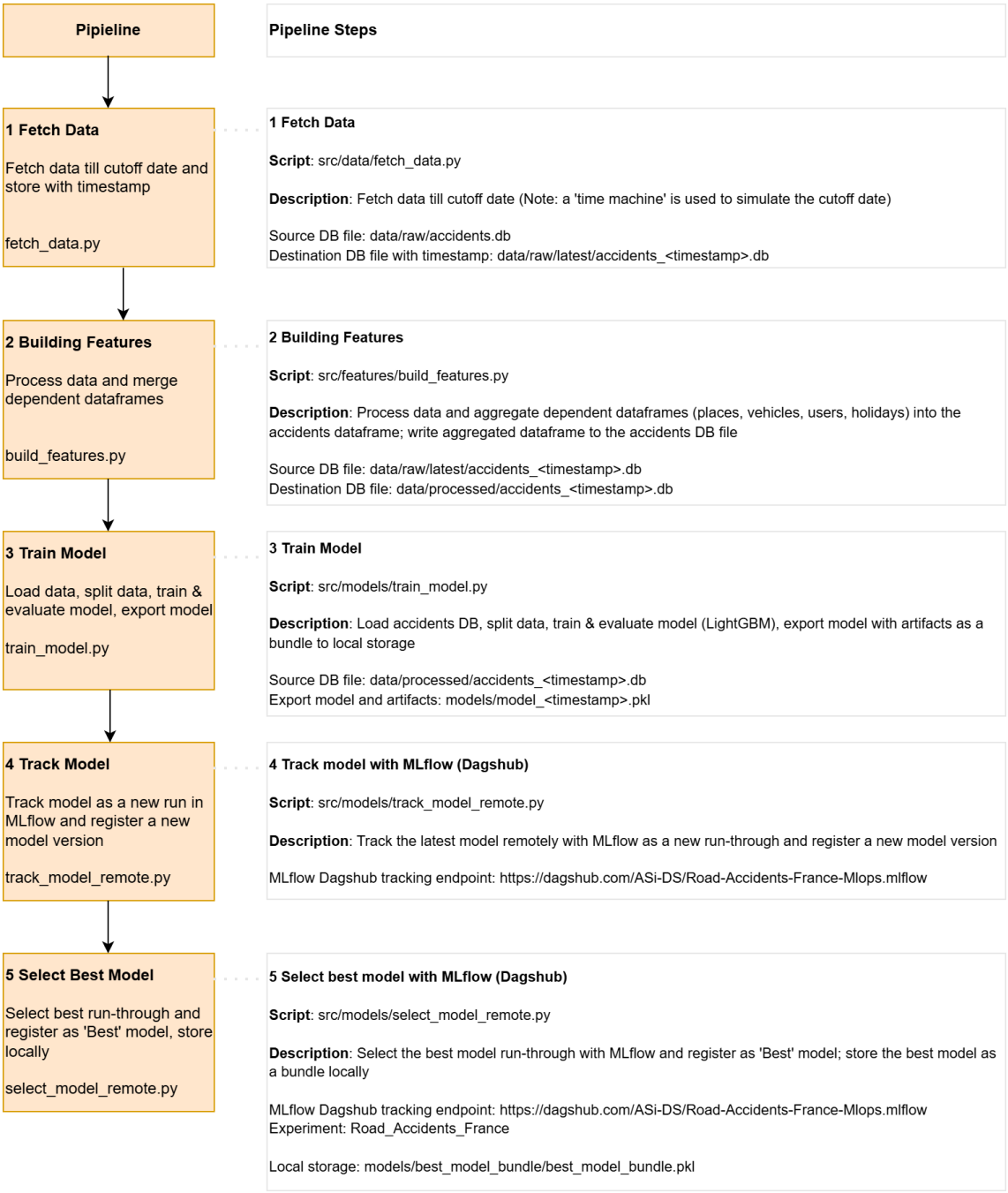
3 Scheduler and Pipeline

The pipeline is executed by a scheduler which runs every 5 min (or as indicated).

The scheduler serves to automate the workflow and makes sure that tasks are exectued in a defined order at specific intervals. Output and errors are logged to a dedicated scheduler.log

Alternatively, the pipeline can also be excuted via an API method: http://localhost:8001/run_pipeline

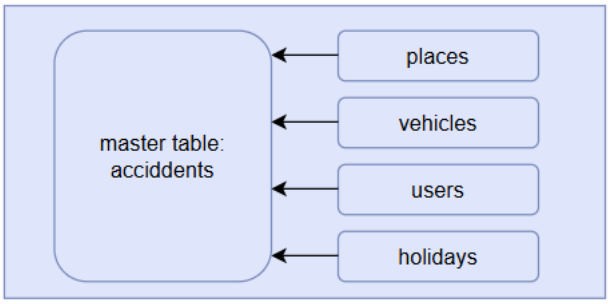
The pipeline consists of these steps:



4 Data loading and processing

Raw data consist of 5 dataframes, resp. tables:

- Accidents (master)
- Places
- Vehicles
- Users
- Holidays



Data loading and processing consists of several steps:

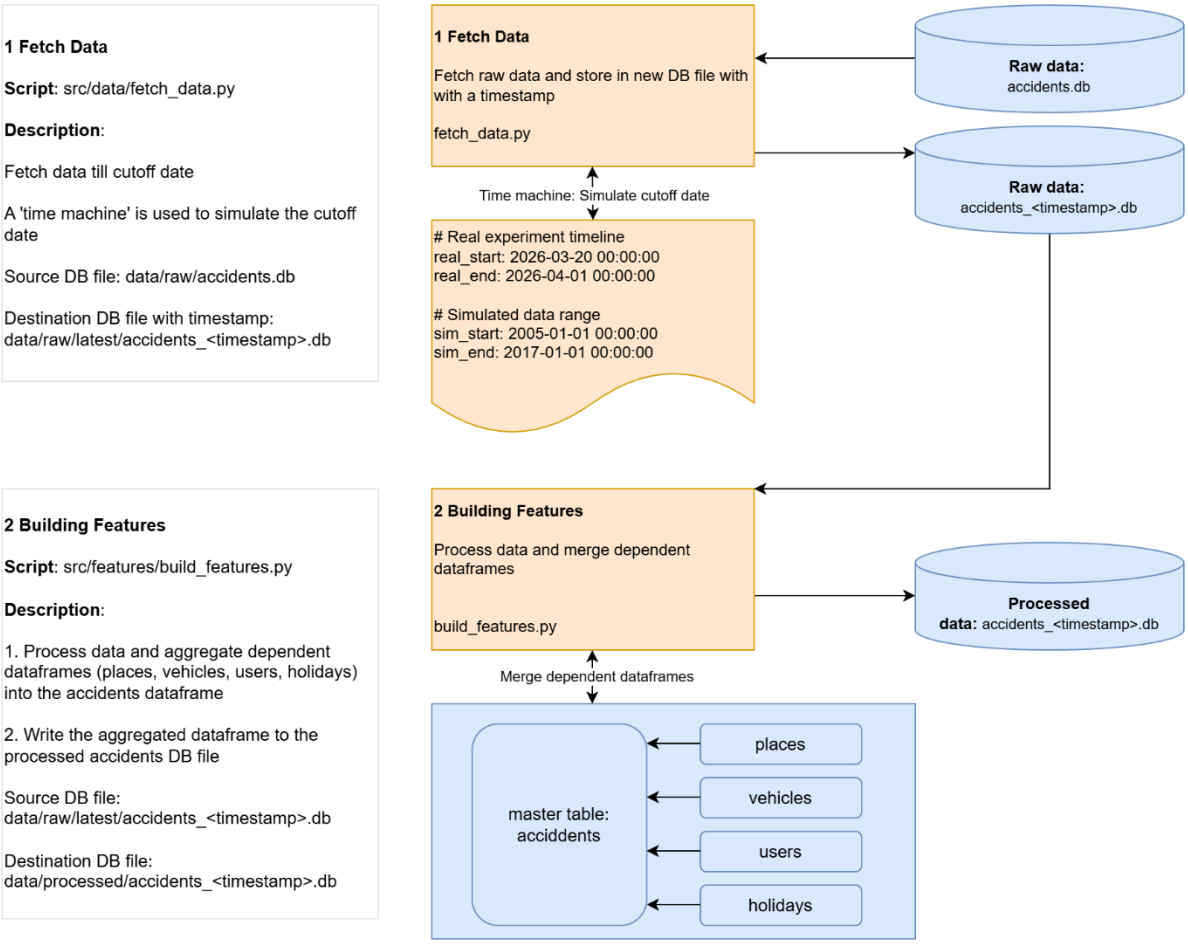
1 Fetch Data

- First, raw data are fetched from a raw DB file till a cut-off data:
  Since no new data are available, the full dataset (stored locally) is accessed till a cut-off date; the cut-off date is calculated using a time machine (s. below).
- The selected data are then written to a new DB file with the timestamp of the cut-off date:
  data/raw/accidents_<timestamp>.db

2 Build Features

- Data are being cleaned (eg. null rows are being dropped) and aggregated:
  Dependent dataframes (Places, Vehicles, Users, and Holidays) are merged into the Accidents dataframe
- The processed and aggregated data are then stored locally to another DB file:
- data/processed/accidents_<timestamp>.db

5 Data Processing Diagram:



Time Machine:

To simulate the cut-off date a 'Time Machine' is being used:

This concept works by proportionally mapping the real-world timeframe of the experiment to a simulated timeframe. It ensures the experiment progresses at the same relative pace, but within a different date range.

Logic:

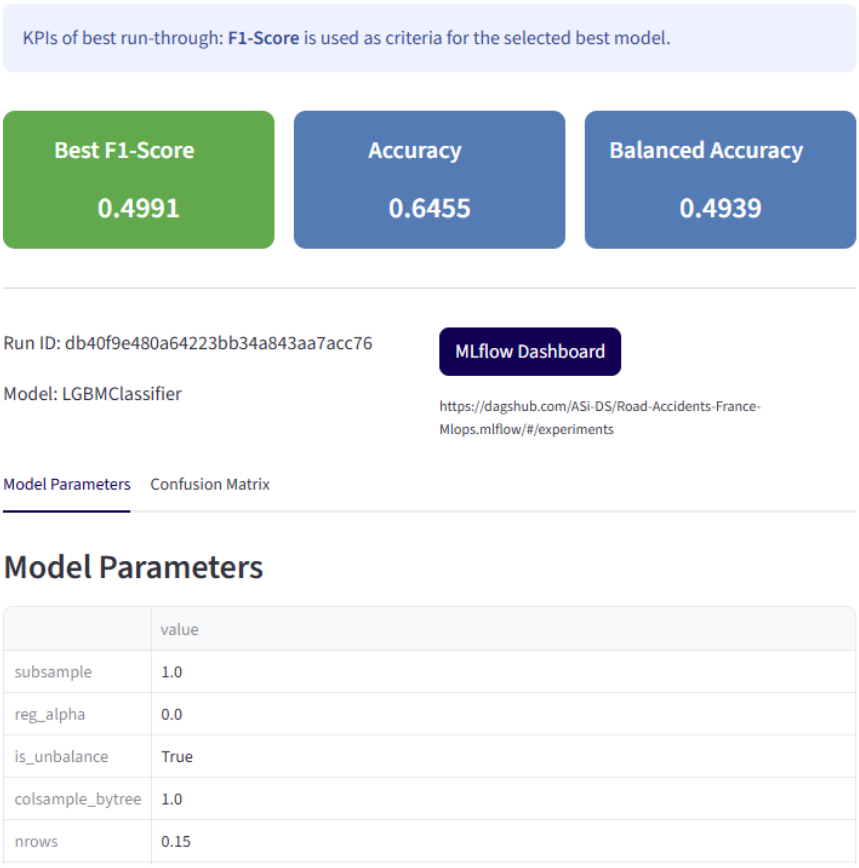
- **Normalization (Ratio):** Calculates progress of the real experiment duration (e.g. halfway between real_start and real_end corresponds to a ratio of 0.5).
- **Linear Mapping:** Applies the same percentage to the simulated duration. If the ratio is 0.5, it returns a timestamp halfway between sim_start and sim_end.
- **DB Filtering (Cut-off):** In the MLOps project, sim_time acts as a moving cut-off point. By querying the database for records where timestamp <= sim_time, "the present" is simulated in the past, hiding data that "hasn't happened yet" in the simulation.
- **In short:** It translates "How much time has passed since I started the script?" into "What date would it be in the historical dataset?"

6 MLflow with DagsHub: Experiment tracking & Model versioning

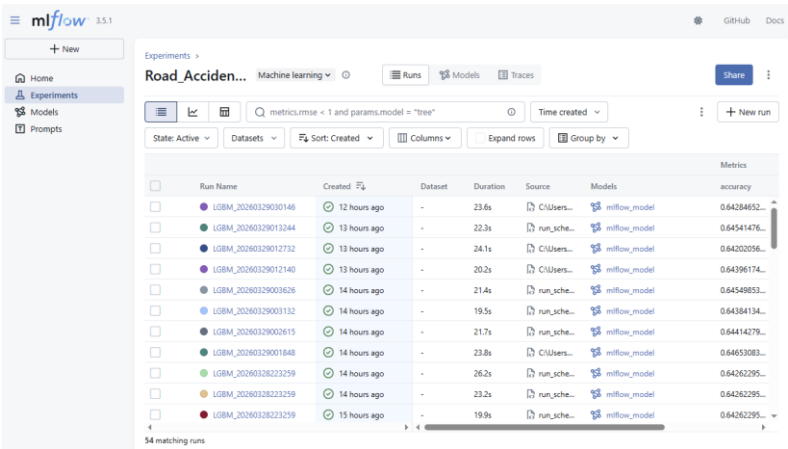
To track experiment run-throughs remotely, MLflow is used together with DagsHub.

The advantage of using DagsHub for remote tracking is the integration with Git, the storage of experiments and registered models remotely, and a web-based UI to visualize MLflow experiment runs.

Example of the MLflow dashboard in the Streamlit presentation:



Dagshub UI: https://dagshub.com/ASi-DS/Road-Accidents-France-Mlops.mlflow

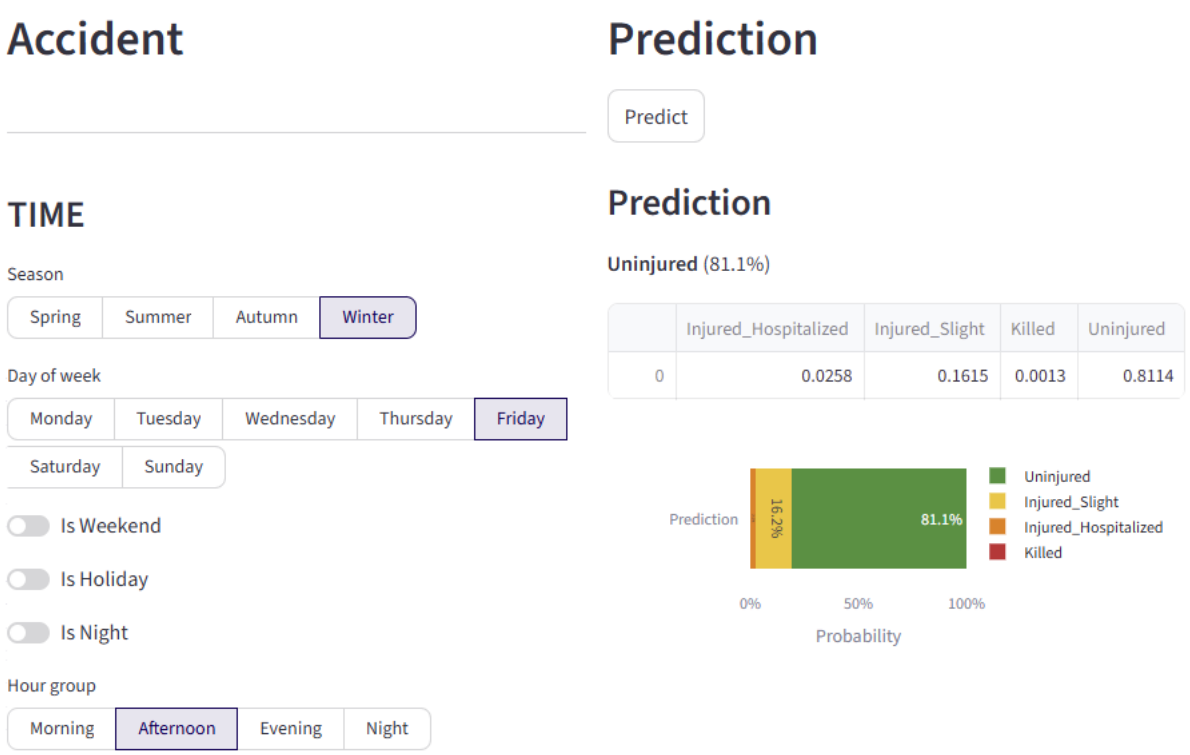


7 Accident Prediction

The accident prediction uses the model to predict the outcome of an accident, based on features in the dataframe.

It can be called via an API method: http://localhost:8001/predict

Example of a prediction, Streamlit screenshot:



8 FastAPI Server Interface

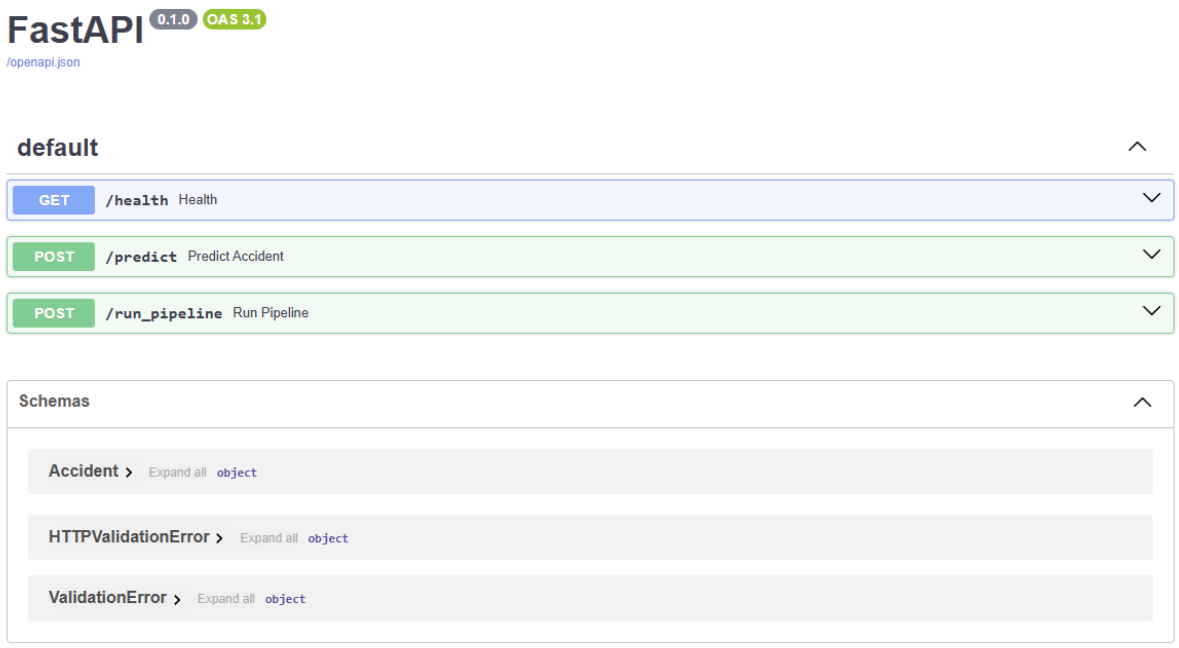
FastAPI provides a convenient deployment layer to implement a microservice architecture.

It wraps service methods into a web API and provides endpoints for clients to interact with the model. Thus, it decouples the service from clients by providing an API contract.

Local URL: http://localhost:8001/docs#/default

API methods currently deployed:

- Health
- Run_Pipeline
- Predict



9 Docker

Docker is a convenient method to isolate and virtualize an application. It creates consistency across different environments and packages the model, code and libraries into a portable unit.

These service components are currently included in docker-compose.yml

API	build:
	  context: .
	  dockerfile: Dockerfile.api
	ports:
	  - "8001:8001"
	volumes:
	  - ./models:/app/models
	environment:
	  MODEL_DIR: /app/models

Streamlit	build:
	  context: .
	  dockerfile: Dockerfile.streamlit
	ports:
	  - "8501:8501"
	environment:
	  API_BASE_URL: http://api:8001
	depends_on:
	  - api

Scheduler	build:
	  context: .
	  dockerfile: Dockerfile.api
	volumes:
	  - ./data:/app/data
	  - ./models:/app/models
	  - ./mlruns:/app/data/mlflow
	depends_on:
	  - api
	command: >
	  uv run python -m scripts.run_scheduler

10 Next Steps & Future Enhancements

Here are some suggestions for further improvements to this project:

1. Infrastructure & Security

- **Scheduling**: Replace the current scheduler script with an orchestrator like *Airflow*. This would allow for automatic retries and provide a visual interface to monitor pipeline health.
- **API Security**: Implement *OAuth2* or *API Key authentication* in FastAPI to ensure that only authorized services can trigger retraining or access the prediction engine.
- **Database Migration**: Transition from SQLite to a production-grade database like *PostgreSQL* for better concurrency and scalability.

2. Automated Testing & CI/CD Integration

- **Unit Testing for API Logic**: Implement *pytest* to automatically verify that endpoints handle data correctly.
- **Automated Testing in CI/CD**: Set up *GitHub Actions* to run a test suite automatically on every push to prevent "broken" code from reaching production.
- **CI/CD for Docker**: Automate the *Docker* build and deployment process using *GitHub Actions*. Every time the code changes, the API and Scheduler containers could be automatically built, tested, and updated.

3. Model Management (ML Life Cycle)

- **Dynamic Training Pipeline**: Shift from a "hard-coded" best model (based on the previous Data Science project) to a flexible training module.
- **Experiment Tracking**: Use *MLflow* to track and compare hyperparameter sweeps and different model architectures within the pipeline.